

Minimum Vertex Cover: a Greedy 2-Approximation, Analysed and Measured

Furkan Reha Tutas

September 2026

Abstract

Minimum Vertex Cover is NP-complete, so exact solutions are out of reach for anything but small graphs. This report studies the classical maximal-matching greedy algorithm: it runs in $O(V + E)$ and never returns a cover more than twice the optimum. We give the NP-completeness reduction from 3-SAT, prove the approximation bound, and then measure the algorithm with a fresh C++17 implementation: validity on 10,000 random graphs with up to 10,000 vertices, approximation quality against an exact solver on graphs of 5–20 vertices, and running time on graphs with up to 200,000 vertices and one million edges. The greedy cover is on average 25–30,% larger than optimal, hits the worst-case ratio of 2 at every size tested, and its running time is linear in V and, in practice, close to constant in E at fixed V . The report closes with three bugs found in the 2019 version of the code that the original experiments could not detect, and how the new test suite catches them.

Contents

1	Problem	1
2	Vertex Cover is NP-complete	2
3	The greedy algorithm	3
4	Exact solver for ground truth	4
5	Experiments	4
6	What the 2019 version got wrong	7
7	Conclusion	7
8	Reproducing	7

1 Problem

Let $G = (V, E)$ be a finite, simple, undirected graph. A **vertex cover** is a set $C \subseteq V$ such that every edge $\{u, v\} \in E$ has $u \in C$ or $v \in C$. The **decision problem** asks, given G and an integer k , whether a vertex cover of size at most k exists. The **optimisation problem** asks for a cover of minimum size, written $\text{OPT}(G)$.

A concrete instance: place the fewest possible traffic cameras at intersections so that every road segment is watched. Roads are edges, intersections are vertices, and a camera at an intersection “covers” every road that meets it.

2 Vertex Cover is NP-complete

2.1 Membership in NP

A certificate is the set C itself. To check it, mark every vertex of C , then scan the edge list once and fail on the first edge with neither endpoint marked. This is $O(V + E)$, so Vertex Cover is in NP.

2.2 Reduction from 3-SAT

Let φ be a 3-CNF formula with L variables and M clauses. Build a graph G_φ and a bound k as follows.

1. **Literal gadgets.** For every variable x_i add two vertices x_i and $\neg x_i$ joined by an edge. Any cover must contain at least one of them.
2. **Clause gadgets.** For every clause $(\ell_1 \vee \ell_2 \vee \ell_3)$ add a triangle with one vertex per literal occurrence. Any cover must contain at least two of the three triangle vertices (one vertex leaves an edge of the triangle uncovered).
3. **Cross edges.** Join each triangle vertex to the literal-gadget vertex carrying the same literal.
4. Set $k = L + 2M$.

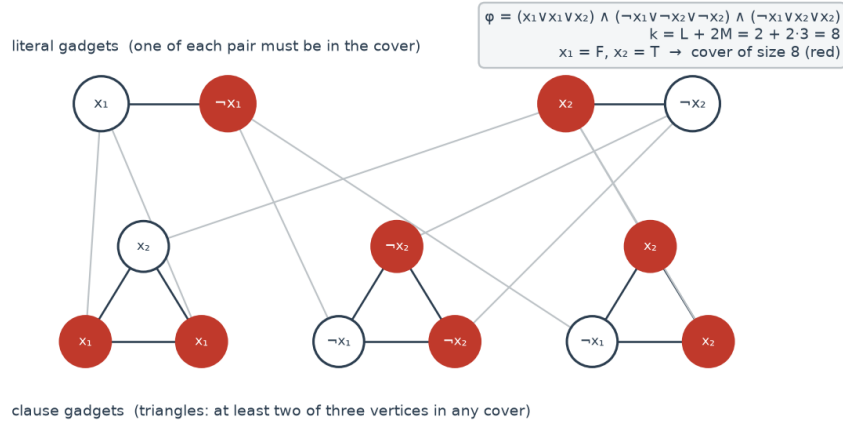


Figure 1: Reduction of $\varphi = (x_1 \vee x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_2) \wedge (\neg x_1 \vee x_2 \vee x_2)$. Red vertices form a cover of size $k = 8$ that encodes the satisfying assignment $x_1 = F, x_2 = T$.

Claim. φ is satisfiable $\iff G_\varphi$ has a vertex cover of size $\leq k$.

($\Rightarrow$) Fix a satisfying assignment. Put the L true literal vertices in the cover; this covers every literal-gadget edge and every cross edge that leaves a true literal. Each clause has at least one true literal, so at least one triangle vertex already has its cross edge covered. Put the *other two* triangle vertices in the cover: this covers the three triangle edges and the remaining two cross edges. Total: $L + 2M = k$.

($\Leftarrow$) Take a cover of size $\leq k$. The L literal edges are disjoint, so at least L cover vertices sit in literal gadgets; the M triangles are disjoint, so at least $2M$ cover vertices sit in triangles. That accounts for all k , hence *exactly* one vertex per literal pair (defining an assignment) and *exactly* two per triangle. The third triangle vertex is uncovered, so its cross edge must be covered from the literal side, which means the corresponding literal is true. Every clause therefore has a true literal.

The construction has $2L+3M$ vertices and $L+3M+3M$ edges, built in one pass over φ . Combined with NP membership, Vertex Cover is NP-complete.

3 The greedy algorithm

Because no polynomial exact algorithm is known, we accept a slightly larger cover in exchange for a guaranteed running time.

3.1 Algorithm

```
C := {}
while there is an edge {u, v} with u not in C and v not in C:
    C := C + {u, v}
return C
```

Equivalently: compute any *maximal matching* and return the endpoints of its edges. The C++ implementation walks each vertex once and stops scanning its adjacency list at the first uncovered neighbour:

```
std::vector<int> greedyCover(const Graph& g) {
    const int n = g.vertexCount();
    std::vector<char> inCover(n, 0);
    std::vector<int> cover;
    for (int u = 0; u < n; ++u) {
        if (inCover[u]) continue;
        for (int v : g.neighbors(u)) {
            if (inCover[v]) continue;
            inCover[u] = inCover[v] = 1;
            cover.push_back(u);
            cover.push_back(v);
            break;
        }
    }
    return cover;
}
```

3.2 Correctness

Suppose the output C misses an edge $\{u, v\}$, i.e. $u, v \notin C$. When the outer loop reached u , u was not in C , and v was in u 's adjacency list and not in C (vertices are never removed from C). The inner loop would therefore have added both. Contradiction, so C is a vertex cover.

3.3 Running time

The outer loop visits each vertex once. The inner loop over u 's neighbours either stops at the first uncovered one or exhausts the list, so across the whole run each adjacency-list entry is examined at most once from each side. Total $O(V + E)$, with $O(V)$ extra memory.

3.4 Approximation ratio

Let A be the set of edges $\{u, v\}$ the algorithm picked. No two edges in A share an endpoint (both endpoints join C the moment an edge is picked, and C only grows), so A is a matching and $|C| = 2|A|$. Any vertex cover, in particular an optimal one, must contain at least one endpoint of every edge in A , and those endpoints are distinct. Hence $|\text{OPT}| \geq |A|$ and

$$|C| = 2|A| \leq 2|\text{OPT}|.$$

The bound is tight: on the star $K_{1,n}$ the algorithm picks one edge and returns 2 vertices, while the centre alone is a cover.

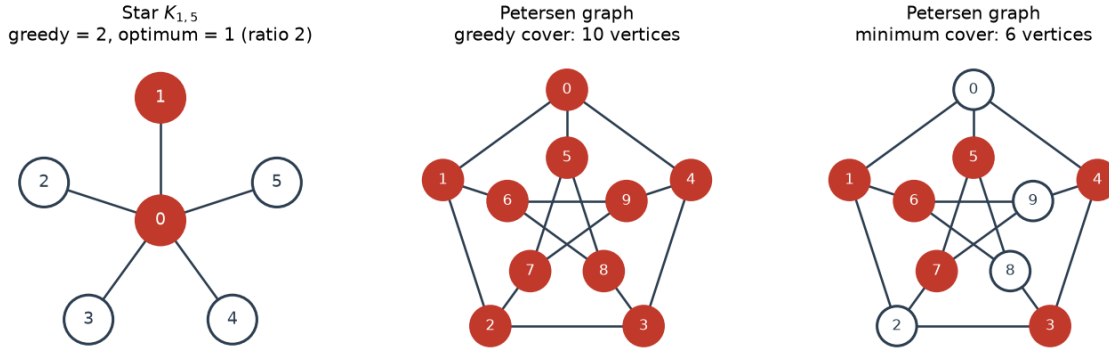


Figure 2: Greedy versus optimum. Left: the star shows the ratio-2 worst case. Centre and right: on the Petersen graph greedy returns all 10 vertices where 6 suffice (ratio 5/3). Output of `vertex_cover solve --file examples/petersen.txt --exact`.

4 Exact solver for ground truth

To measure how far greedy is from optimal we need OPT on small graphs. `exactCover` enumerates vertex subsets as 64-bit masks in order of increasing population count (Gosper's hack yields the next mask with the same number of set bits) and returns the first subset that intersects every edge's two-bit mask. This is $O(\sum_k \binom{V}{k} \cdot E)$ in the worst case and is used only for $V \leq 20$.

5 Experiments

All numbers below are reproducible with the commands shown, seed 42, on the implementation in this repository (GCC 13, -O3, Linux container on a desktop CPU).

5.1 Validity

```
vertex_cover verify 10000 1000 10000 20000
valid covers: 10000/10000
```

10,000 random graphs, $V \sim U[1000, 10000]$, $E \sim U[V, 20000]$, every vertex of degree ≥ 1 . Every output passed `isVertexCover`, which checks each edge directly. The same property is enforced as a `cctest` (200 graphs) on every push.

5.2 Approximation quality

Define **quality** $q = |\text{OPT}|/|C| \in [0.5, 1]$, so $q = 1$ means greedy found an optimum and $q = 0.5$ is the worst case allowed by the bound. For each $V \in \{5, \dots, 20\}$ we draw 500 graphs with $E \sim U[V-1, \binom{V}{2}]$.

Table 1: `vertex_cover` quality 500. Quality is the mean over 500 graphs per size.

V	mean q	worst ratio $ C / \text{OPT} $	greedy = OPT
5	0.738	2.0	81 / 500
6	0.678	2.0	39 / 500
7	0.734	2.0	49 / 500
8	0.699	2.0	16 / 500
9	0.730	2.0	19 / 500
10	0.723	2.0	14 / 500
12	0.740	2.0	9 / 500
15	0.775	2.0	9 / 500
18	0.765	2.0	0 / 500
20	0.782	2.0	2 / 500

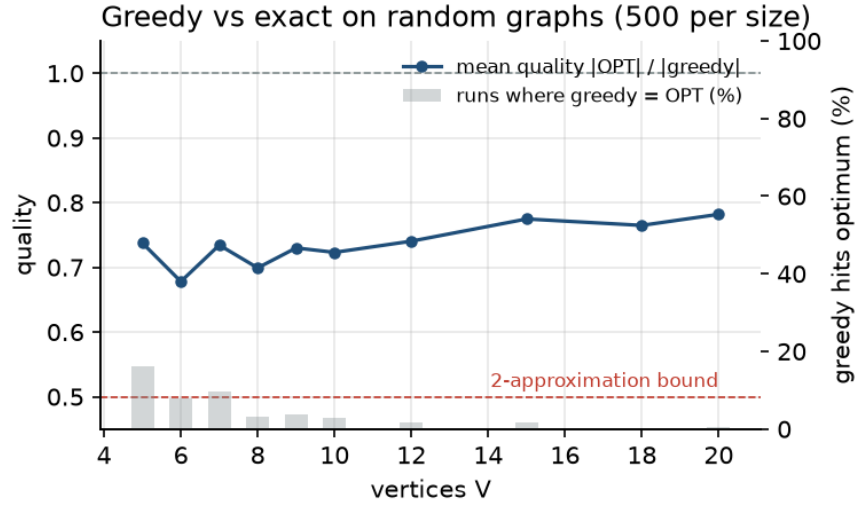
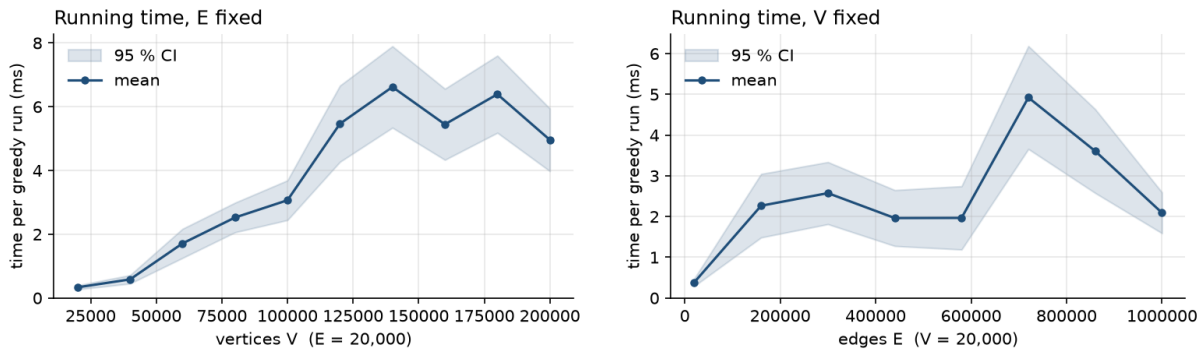


Figure 3: Mean quality (line, left axis) and share of runs where greedy matched the optimum (bars, right axis).

Three observations. First, the mean quality sits at 0.70–0.78, i.e. the greedy cover is typically 25–30,% larger than optimal, well inside the factor-2 guarantee. Second, the worst case *is* reached at every size: random sparse graphs regularly contain star-like pieces where greedy spends two vertices on one. Third, the chance of hitting the exact optimum collapses as V grows, from 16,% at $V = 5$ to under 1,% at $V \geq 18$. Greedy is a good bound, not a good exact heuristic.

5.3 Running time

Sizes were chosen so a single greedy run takes milliseconds rather than microseconds. Each point is the mean of 50 runs on 50 independent random graphs with an untimed warm-up run per graph; the band is the 95,% confidence interval ($\bar{t} \pm 1.96 s/\sqrt{n}$).



With E fixed, time grows roughly linearly with V up to about 140,000 vertices and then flattens into noise; the graph is so sparse there that most vertices have degree one and the run is dominated by the sequential scan of `inCover`, which is memory-bound. With V fixed, time stays within 2–5,ms while E grows fifty-fold. That is not a contradiction of $O(V + E)$: the bound is an upper bound on adjacency entries *examined*, and the inner loop stops at the first uncovered neighbour. In a dense

random graph almost every vertex is covered within the first few hundred outer iterations, after which the remaining vertices are skipped in $O(1)$. The worst case ($\Theta(V + E)$) needs adversarial inputs such as long paths with pendant vertices; random graphs are far from it.

Timing in a container on a desktop OS is noisy (the confidence bands are wide even at 50 samples). The direction of the results is robust; the individual means are not to be quoted to more than one significant figure.

6 What the 2019 version got wrong

The original project (Sabancı University CS301, Fall 2019) reached the same conclusions on paper but its code, rewritten here from a single 400-line `main.cpp`, contained three defects that the experiments of the time could not have revealed.

The timed greedy did not return a vertex cover. After picking an edge $\{u, v\}$ it marked *every neighbour* of u and v as visited. A vertex w marked this way was never examined again, so an edge $\{w, x\}$ with x also skipped stayed uncovered. The function used in the correctness test was a separate, correct implementation, so the bug was invisible.

The correctness check tested the wrong property. `isVertexCover` marked the cover and its neighbours and then checked that every *vertex* was marked. That is the definition of a *dominating set*, not a vertex cover; a dominating set can leave edges between two non-dominating vertices uncovered. Since the buggy greedy above happened to produce dominating sets, the check passed 10,000 out of 10,000 times.

The generator could hang and emit self-loops. Edge counts above $\binom{V}{2}$ made the rejection loop spin forever, and the pass that gives every vertex an edge could pick the vertex itself.

The rewrite adds a unit test that asserts, on 300 random graphs, that greedy's output *is a cover according to an edge-by-edge check*, is no smaller than the exact optimum, is at most twice it, and has even size. Any one of the original three bugs fails at least one of these assertions.

7 Conclusion

Vertex Cover is NP-complete, by membership in NP and a linear-size reduction from 3-SAT. The maximal-matching greedy algorithm trades optimality for an $O(V + E)$ bound and a guaranteed factor of 2. Empirically it is about $1.3\times$ optimal on random graphs, reaches the factor-2 worst case routinely, and rarely lands exactly on the optimum. Its running time is linear in V and, on random inputs, nearly independent of E because the scan stops early. Writing the checker correctly turned out to matter more than writing the algorithm correctly: a test that measures the wrong property gives full marks to a wrong answer.

8 Reproducing

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release && cmake --build build
ctest --test-dir build
./build/vertex_cover verify 10000 1000 10000 20000
./build/vertex_cover quality 500 > benchmarks/results/quality-500.csv
./build/vertex_cover bench 50 --out benchmarks/results
```

```
python benchmarks/plot.py           # docs/figures/time-*.png, quality.png
python docs/figures/make_figures.py  # reduction and Petersen illustrations
docker run --rm -v "$PWD:/w" -w /w/docs pandoc/latex:3.6 report.md -o report.pdf
```

The 2019 group report by T. Garib, F. R. Tutaş, D. B. Çakar, M. Arslan and E. H. Songur (instructor H. Yenigün) is preserved unchanged in `docs/legacy/`.